

Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition

Anonymous Author(s)

Abstract

Large language models can generate short RTL fragments, but remain unreliable on benchmark-level hardware designs that require timing, bit-accurate arithmetic, memory protocols, and golden-output agreement. This study evaluates verifier-grounded autonomous RTL synthesis on ArchXBench, a complex digital-systems benchmark whose reported frontier-model results collapse from Level 4 onward under direct prompting. Four conditions are compared: single-shot generation, monolithic golden-feedback repair, decomposition-guided iterative repair, and testbench-localized decomposition. On original ArchXBench contracts, the evaluated pipeline obtains clean solves across Levels 3–6, including robust Level-4 FFT/IFFT and floating-point pipeline solves, Level-5 convolution/image-processing designs, and Level-6 AES encryption/decryption with golden verification. The results also show that decomposition is not universally superior: monolithic golden-feedback repair is a strong baseline on several Level-5 and repaired-contract rows. A second contribution is an executable-contract audit that separates original benchmark solves from minimally repaired contract fixtures. The audit reveals both benchmark defects and genuine unsolved capability boundaries, showing that hard RTL synthesis evaluation requires trustworthy executable contracts in addition to strong generation methods.

CCS Concepts

• **Hardware** → **Hardware description languages and compilation**; **Electronic design automation**; • **Software and its engineering** → *Automatic programming*.

Keywords

RTL synthesis, Verilog generation, LLM agents, CEGIS, hardware verification, benchmark evaluation

ACM Reference Format:

Anonymous Author(s). 2027. Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition. In *Proceedings of 32nd Asia and South Pacific Design Automation Conference (ASP-DAC '27)*. ACM, New York, NY, USA, 5 pages.

1 Introduction

Autonomous code agents are commonly evaluated by an executable loop: generate a candidate, run a checker, inspect failures, and repair. This loop is appealing because it grounds language-model generation in objective feedback. However, the same loop is substantially more difficult when the target artifact is a register-transfer level (RTL) hardware design rather than a short software function. RTL behavior depends on bit widths, signedness, clocking, reset protocols, pipeline latency, streaming handshakes, and external file-output conventions. A design may compile while still being

functionally wrong; a simulator may run to completion while still producing incorrect golden traces.

This work studies this problem on ArchXBench, a recent benchmark suite for complex RTL synthesis. Unlike VerilogEval [3] and RTLLM [4], which established automatic evaluation for Verilog generation on smaller designs, ArchXBench includes multi-level tasks spanning arithmetic kernels, FFTs, filters, image processing, matrix multiplication, and cryptography [7]. The ArchXBench paper reports that frontier models solve lower-level tasks but fail from Level 4 onward under zero-shot pass@5 evaluation. This failure frontier makes ArchXBench a useful stress test for autonomous RTL synthesis.

The central question addressed in this study is: *can verifier-grounded autonomous synthesis solve hard RTL benchmark tasks that direct generation does not reliably solve, while preserving a clear distinction between model failures and benchmark-contract failures?*

The answer is partially affirmative, with important caveats. Four conditions are evaluated: a single-shot baseline (C1), a monolithic golden-feedback repair baseline (C2g), a decomposition-guided iterative repair pipeline (C4i), and a testbench-localized decomposition pipeline (C4tl). C4i and C4tl solve several rows that direct prompting does not solve, including hard Level-4 and golden-verified Level-5/Level-6 tasks. At the same time, C2g is a strong baseline and matches or exceeds decomposition on several rows. The contribution is therefore not a claim that modular decomposition dominates monolithic repair. Rather, the study shows that verifier-grounded repair and decomposition can close important hard-task gaps, and that benchmark contracts must be audited before successes or failures are interpreted.

This paper makes three contributions:

- (1) A verifier-grounded autonomous RTL synthesis pipeline combining iterative repair, modular decomposition, and simulator/golden feedback.
- (2) A controlled ArchXBench Level 3–6 evaluation that separates original-contract results, repaired-contract results, held rows, and diagnostic failures.
- (3) A benchmark-contract audit documenting executable evaluation defects in hard RTL tasks and showing how these defects change the interpretation of synthesis results.

2 Related Work

2.1 LLM-Based RTL Generation

VerilogEval introduced an HDLBits-style benchmark for Verilog generation and enabled automatic functional checking of generated RTL [3]. RTLLM similarly proposed an open benchmark for design RTL generation with large language models [4]. These benchmarks were essential for making RTL generation measurable, but many tasks are small instructional circuits or moderate fragments rather than deeply structured systems.

More recent work moves beyond direct prompting. AutoChip uses compiler and simulator feedback to improve HDL generation [9]. VerilogCoder introduces graph-based planning and AST-based waveform tracing for autonomous Verilog coding [1]. AutoVeriFix uses simulation discrepancies to correct LLM-generated Verilog [8]. Spec2RTL-Agent studies LLM-agent systems for complex specification-to-RTL generation [11]. VFlow searches over agentic workflows for Verilog generation using Monte Carlo tree search [10]. HDLxGraph uses HDL-specific graph retrieval to support repository-level HDL tasks [12]. Evolutionary methods such as REvolution, Evolve, and COEVO use inference-time search or co-evolutionary optimization for RTL correctness and, in some cases, power/performance/area optimization [2, 5, 6].

This study differs in emphasis. It evaluates on ArchXBench Levels 3–6, including rows at and above the reported Level-4 failure frontier. It also separates original benchmark contracts from repaired executable contracts. This distinction is important because an RTL benchmark can fail because the generated design is wrong, but it can also fail because the released testbench, file format, or golden comparator is inconsistent.

Table 1 summarizes the positioning. Prior work has made substantial progress on LLM-aided RTL generation, repair, workflow search, and retrieval. The present study is complementary: it focuses on hard ArchXBench rows and on the reliability of the executable contracts used to judge those rows.

2.2 Verifier-Grounded Synthesis

Counterexample-guided inductive synthesis (CEGIS) uses a verifier to reject incorrect candidates and guide the next synthesis attempt. LLM-based repair loops instantiate a related pattern: generate code, run tests, inspect errors, and repair. For RTL, however, verifier feedback is heterogeneous. Failures can arise from syntax, elaboration, simulation, timing, protocol mismatch, file-output mismatch, or incorrect golden references. The present pipeline treats simulator and golden-output feedback as first-class signals, but also audits the benchmark contract when the executable artifacts disagree.

3 Problem Setting

Each ArchXBench design provides a natural-language specification and executable assets such as Verilog testbenches, input files, output files, and comparison scripts. An autonomous synthesis condition receives the task and emits RTL. For self-checking designs, a run is successful only when the official testbench passes all checks. For file-output designs, simulator completion is not sufficient; the generated output must match the golden output.

Three evaluation classes are distinguished throughout the paper:

- **Original-contract results:** runs against the released benchmark as-is.
- **Repaired-contract results:** runs against a separate copied fixture in which infrastructure or executable-contract inconsistencies are minimally repaired.
- **Held or excluded rows:** tasks for which the released contract remains ambiguous enough that a positive solve would be misleading.

This separation is necessary for credible RTL evaluation. If a checker is wrong, passing it may not mean solving the intended

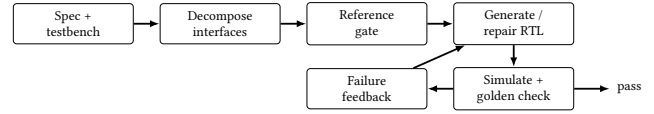


Figure 1: Verifier-grounded decomposition and repair loop. The decomposition is useful only after a reference composition passes the system-level checker.

hardware task. If a checker cannot compare outputs correctly, failing it may not mean the model failed.

Table 2 gives the contract taxonomy used during the audit. The taxonomy is intentionally operational: each category changes how a run is interpreted. For example, a self-checking testbench can directly define a solve, while a display-only checker cannot.

4 Method

4.1 Evaluated Conditions

C1: single-shot generation. The model receives the task and emits RTL without repair. This condition measures whether the task is already solved by direct prompting.

C2g: monolithic golden-feedback repair. The model generates a complete design, the simulator and golden checker evaluate it, and the model receives full-design feedback for repair. The artifact remains monolithic throughout the loop. This is a strong baseline because it uses the same external correctness signal without introducing module-boundary risk.

C4i: decomposition-guided iterative repair. The model decomposes the design into submodules with interfaces and a top-level composition. The reference composition must pass the system testbench before the decomposition is used as synthesis scaffolding. This validation gate is intended to prevent plausible-looking but ungrounded decompositions. After validation, modules are implemented and repaired under system feedback.

C4tl: localized decomposition. C4tl extends decomposition-guided repair with more localized failure information from simulation and testbench traces. The goal is to reduce repair diffusion by identifying which module or interface is most implicated by the failing behavior.

4.2 Artifact and Verification Discipline

Each run is recorded under the repository artifact tree with a result file. Artifact-backed rows include generated RTL and, where applicable, decomposition metadata. File-output designs require strict golden verification; native simulator completion alone is treated as diagnostic evidence. Trusted score-only rows are labeled separately and are not described as artifact-backed.

5 Experimental Setup

5.1 Benchmark Scope

The evaluation covers ArchXBench Levels 3–6. Lower levels are not emphasized because the benchmark paper already reports that current models can solve lower-level tasks. The central interest is

Table 1: Positioning relative to recent LLM-aided RTL and verification work. The distinguishing focus of this work is hard ArchXBench L3–L6 evaluation with explicit original-contract versus repaired-contract accounting.

Work	Primary target	Feedback / search	Main benchmark style	Difference from this study
VerilogEval / RTLLM	RTL generation benchmarks	Testbench evaluation	HDLBits-style / moderate RTL	Establishes automatic evaluation, but mostly easier than ArchXBench L4–L6
AutoChip / AutoVeriFix	HDL generation and repair	Compiler, simulator, oracle feedback	Verilog repair/generation tasks	Iterative correction is related; benchmark-contract audit is not central
VerilogCoder / Spec2RTL-Agent	Autonomous RTL agents	Planning, tracing, multi-agent generation	VerilogEval / RTLLM-style tasks	Strong agent baselines; not focused on ArchXBench hard-level contract failures
VFlow	Workflow search	MCTS over agent workflows	Verilog generation tasks	Searches agent workflows; does not study executable-contract validity
REvolution / Evolve / COEVO	Evolutionary RTL search	Population search and PPA/correctness feedback	VerilogEval / RTLLM-style tasks	Search-oriented; this study emphasizes ArchXBench hard rows and benchmark contracts
This work	Hard RTL synthesis	Golden repair, decomposition, contract audit	ArchXBench L3–L6	Separates original solves, repaired-contract solves, and held rows

Table 2: Executable-contract taxonomy used to classify ArchXBench rows.

Contract type	Paper treatment
Self-checking testbench	Pass all official checks; native pass score is sufficient.
File-output with golden	Require post-simulation golden comparison; simulation completion is diagnostic only.
Display-only checker	Not a solve until expected values are converted into executable checks.
Stale comparator or schema mismatch	Repair only in a separate copied fixture and report separately.
Spec/testbench disagreement	Exclude or hold unless a principled source of truth is available.
Missing oracle	Hold; do not claim positive results.

Table 3: Practical profile across repo-local gpt-5.5 artifacts. Wall time is measured on local runs and is hardware-dependent.

Cond.	Runs	Clean	Avg. calls	Avg. wall
C1	100	16	5.0	9.2 min
C2g	69	41	15.5	22.6 min
C4i	125	76	14.6	14.8 min
C4tl	143	83	7.4	9.3 min

the harder regime that includes iterative arithmetic, FFT/IFFT designs, floating-point pipelines, FIR filters, image-processing kernels, matrix multiplication, and AES.

5.2 Seeds, Model, and Metrics

Main paper tables use seeds 42, 123, and 456. Two additional C4tl Level-4 seeds, 789 and 1024, are used as robustness evidence. The primary model label is gpt-5.5, as recorded in the repository artifacts.

For self-checking designs, a clean solve requires `best_passes == total_tests` with `total_tests > 0`. For file-output designs, a clean solve requires `golden_correct == golden_total` with `golden_total > 0`. Solve rates are reported over seeds.

Table 3 reports a practical profile from repo-local gpt-5.5 artifacts. These values are not used as a primary efficiency benchmark because the rows include heterogeneous designs, but they expose the engineering tradeoff: C2g and C4i are generally more expensive, while C4tl is lighter but less robust on several rows.

6 Results on Original ArchXBench Contracts

Table 4 summarizes the main original-contract results. The strongest Level-3 method-value evidence is produced by C4i, which solves five selected Level-3 rows on all three seeds while C1 and C2g fail or nearly fail on those rows. The strongest hard-level coverage evidence is produced by C4tl on Level 4: the FFT/IFFT and floating-point pipeline rows solve on all three main seeds, and remain solved on two additional robustness seeds. At Levels 5 and 6, golden-verified solves are obtained on convolution/image-processing and AES rows.

The results are deliberately not presented as a universal decomposition win. C2g is a serious baseline: it solves Level-4 FFT/IFFT rows, several Level-5 rows, and multiple repaired-contract rows. The most accurate interpretation is that decomposition provides value on specific hard rows and gives a structured synthesis process, while monolithic golden-feedback repair is often sufficient and sometimes better.

7 Executable-Contract Audit

Hard RTL benchmarks are useful only when their executable contracts reflect the intended task. During evaluation, several rows were found to contain inconsistent specifications, stale comparators, display-only checkers, hidden assumptions, or output-schema mismatches. The released benchmark is not overwritten. Instead, repaired fixtures are kept under a separate artifact root and are reported separately from original-contract results.

Table 5 reports the repaired-contract evidence. These rows are not counted as original ArchXBench solves. They answer a narrower question: when an infrastructure or executable-contract inconsistency is minimally repaired and oracle-validated, do the synthesis agents solve the intended task?

The audit produces both positive and negative evidence. For `multich_conv2d` and `quantized_matmul`, repairing the executable contract makes the intended task measurable and solvable. For `systolic_gemm`, converting display-only expected matrices into executable checks does not produce a win; all methods still fail. This is an important negative result because it shows that the audit is not merely a mechanism for converting failures into successes.

Case study: display-only GEMM checking. The released `systolic_gemm` checker prints expected matrices but does not reliably convert them into machine-readable pass/fail decisions. The repaired-contract fixture uses the expected matrices already present

Table 4: Original-contract results on ArchXBench. Values are clean solves over seeds. GV denotes golden-verified file-output rows.

Level	Design	C1	C2g	C4i	C4tl	Evidence / interpretation
L3	fp_adder	0/5	1/5	3/3	3/3	C4i/C4tl improve over direct generation
L3	fp_multiplier	0/5	0/3	3/3	3/3	clean decomposition-guided solve
L3	gauss_siedel	0/3	0/3	3/3	1/3	C4i stronger than C4tl
L3	gradient_descent	0/3	0/3	3/3	3/3	clean decomposition-guided solve
L3	newton_raphson_sqrt	0/3	0/3	3/3	3/3	clean decomposition-guided solve
L3	newton_raphson_polynomial	0/3	0/3	0/3	0/3	original checker diagnostic
L4	fft_16pt_iterative	0/3	3/3	0/3	3/3; 5/5	hard-row solve; robustness seeds for C4tl
L4	ifft_16pt_iterative	0/3	3/3	0/3	3/3; 5/5	hard-row solve; robustness seeds for C4tl
L4	fp_adder_pipeline	3/3	3/3	3/3	3/3; 5/5	all methods solve
L4	fp_mult_pipeline	3/3	3/3	3/3	3/3; 5/5	all methods solve
L5	conv1d	3/3	3/3	3/3	1/3	GV; C4i clean
L5	conv2d	0/3	3/3	0/3	0/3	GV; monolithic repair strongest
L5	dct_idct_8pt_pipelined	0/3	3/3	0/3	0/3	GV; monolithic repair strongest
L5	harris_corner_detection	0/3	3/3	3/3	0/3	GV; C4i matches C2g
L5	unsharp_mask	0/3	3/3	0/3	0/3	GV; artifact-backed C2g
L6	aes_encryption	3/3	3/3	3/3	2/3	GV; all major conditions strong
L6	aes_decryption	1/3	3/3	3/3	1/3	GV; repair improves direct generation

Table 5: Repaired-contract results. These rows are separate from original ArchXBench results.

Design	C2g	C4i	C4tl
conv_3d	3/3	2/3	0/3
multich_conv2d	3/3	3/3	3/3
quantized_matmul	3/3	3/3	0/3
fp_band_pass_fir	3/3	0/1	0/1
fp_high_pass_fir	3/3	0/1	0/1
newton_raphson_polynomial	3/3	1/3	1/3
systolic_gemm	0/3	0/3	0/3

in the testbench to define executable comparisons. After this repair, C2g, C4i, and C4tl all remain at 0/3. This row is therefore reported as a genuine remaining capability boundary, not as a benchmark-repair success.

Several rows remain held or excluded. The L4 FIR family is excluded from positive tables because the specification and executable testbench disagree on filter coefficients/source-of-truth behavior. The L6 fp_low_pass_fir row is held because the released files do not expose an explicit coefficient/cutoff oracle. The L6 fft_streaming_64pt row is excluded because the released input/output contract has unresolved schema and numeric-encoding ambiguities.

8 Discussion

8.1 When Decomposition Helps

Decomposition is most useful when a design naturally factors into modules and when failure localization reduces the repair burden. The Level-4 FFT/IFFT and floating-point pipeline rows support this conclusion: C4tl robustly solves all four core Level-4 rows on the main seeds and two additional robustness seeds. C4i also provides strong Level-3 evidence on numerical kernels where C1 and C2g fail.

The reference-composition gate is essential. A decomposition is not accepted merely because it appears plausible. It must be validated against the system-level checker before it becomes synthesis

scaffolding. This requirement prevents arbitrary module boundaries from becoming a source of false confidence.

8.2 When Decomposition Does Not Help

C4i and C4tl do not dominate C2g. Several Level-5 and repaired-contract rows are solved most cleanly by monolithic golden-feedback repair. This can happen when the design is compact enough for whole-design repair, when module boundaries introduce integration risk, or when the primary difficulty is a global convention such as file format, fixed-point encoding, or streaming protocol. Therefore, the paper’s claim is intentionally conservative: decomposition is a useful synthesis strategy, not a universally superior one.

8.3 Why Benchmark Contracts Matter

The benchmark audit is a design-automation contribution rather than an incidental engineering detail. LLM-aided RTL synthesis papers increasingly rely on executable benchmarks. If the executable contract is ambiguous, a model can fail for the wrong reason or pass the wrong task. The observed issues include hidden testbench assumptions, stale comparators, file-output schema mismatches, display-only checkers, unsatisfiable residual checks, and missing oracle information. Reporting these separately improves the reliability of future comparisons on hard RTL benchmarks.

9 Limitations

First, the evaluated methods do not establish universal dominance over monolithic repair. C2g is strong and should be treated as a primary baseline. Second, some historical rows are trusted score-only rather than artifact-backed; these rows are labeled accordingly in the repository and should not be described as artifact-backed until generated RTL is present. Third, repaired-contract evaluation depends on careful judgment about what constitutes an infrastructure repair rather than a semantic benchmark change. This risk is mitigated by keeping repaired fixtures separate, oracle-validating

repairs when possible, and excluding unresolved rows. Fourth, the empirical study is concentrated on RTL synthesis. The broader lesson may transfer to other executable synthesis domains, but that transfer is not established here.

10 Conclusion

This study evaluated autonomous RTL synthesis on hard ArchXBench designs using iterative repair, modular decomposition, and strict simulator/golden feedback. Clean original-contract solves were obtained across Levels 3–6, including robust Level-4 rows and golden-verified Level-5/Level-6 designs. The results also show that monolithic golden-feedback repair is a strong baseline and that decomposition is not universally superior. A separate executable-contract audit revealed benchmark defects, repaired some infrastructure-level contracts, and identified genuine remaining failures such as `systolic_gemm`.

The main conclusion is that hard autonomous RTL synthesis requires three ingredients together: structured generation, executable feedback, and trustworthy benchmark contracts. Without all three, both successes and failures can be misinterpreted.

References

- [1] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [2] Wei-Po Hsin, Ren-Hao Deng, Yao-Ting Hsieh, En-Ming Huang, and Shih-Hao Hung. 2026. Evolve: Evolutionary Search for LLM-based Verilog Generation and Optimization. *arXiv preprint arXiv:2601.18067* (2026).
- [3] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*.
- [4] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).
- [5] Kyungjun Min, Kyumin Cho, Junhwan Jang, and Seokhyeong Kang. 2026. REvolution: An Evolutionary Framework for RTL Generation Driven by Large Language Models. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [6] Heng Ping, Peiyu Zhang, Shixuan Li, Wei Yang, Anzhe Cheng, Shukai Duan, Xiaole Zhang, and Paul Bogdan. 2026. COEVO: Co-Evolutionary Framework for Joint Functional Correctness and PPA Optimization in LLM-Based RTL Generation. *arXiv preprint arXiv:2604.15001* (2026).
- [7] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A Complex Digital Systems Benchmark Suite for LLM Driven RTL Synthesis. *arXiv preprint arXiv:2508.06047* (2025).
- [8] Yan Tan, Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [9] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. In *Proceedings of the ACM/IEEE Design Automation Conference*.
- [10] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2026. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. In *Proceedings of the Asia and South Pacific Design Automation Conference*.
- [11] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine Lin, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. *arXiv preprint arXiv:2506.13905* (2025).
- [12] Pingqing Zheng, Jiayin Qin, Fuqi Zhang, Shang Wu, Yu Cao, Caiwen Ding, and Yang Zhao. 2025. HDLxGraph: Bridging Large Language Models and HDL Repositories via HDL Graph Databases. *arXiv preprint arXiv:2505.15701* (2025).